Backpropagation with Cross-Entropy Loss

[Draft 6]

Devi Prasad M. *
Manipal School of Information Sciences.

October, 2026

1 The setup

We will use the example network of Figure 1 in this exercise. This is a fully connected network with two hidden layers of width 3. The hidden layers use ReLU activation. The network maps an input $\mathbf{x} \in \mathbb{R}^2$ to a probability vector $\mathbf{q} \in \mathbb{R}^3$.

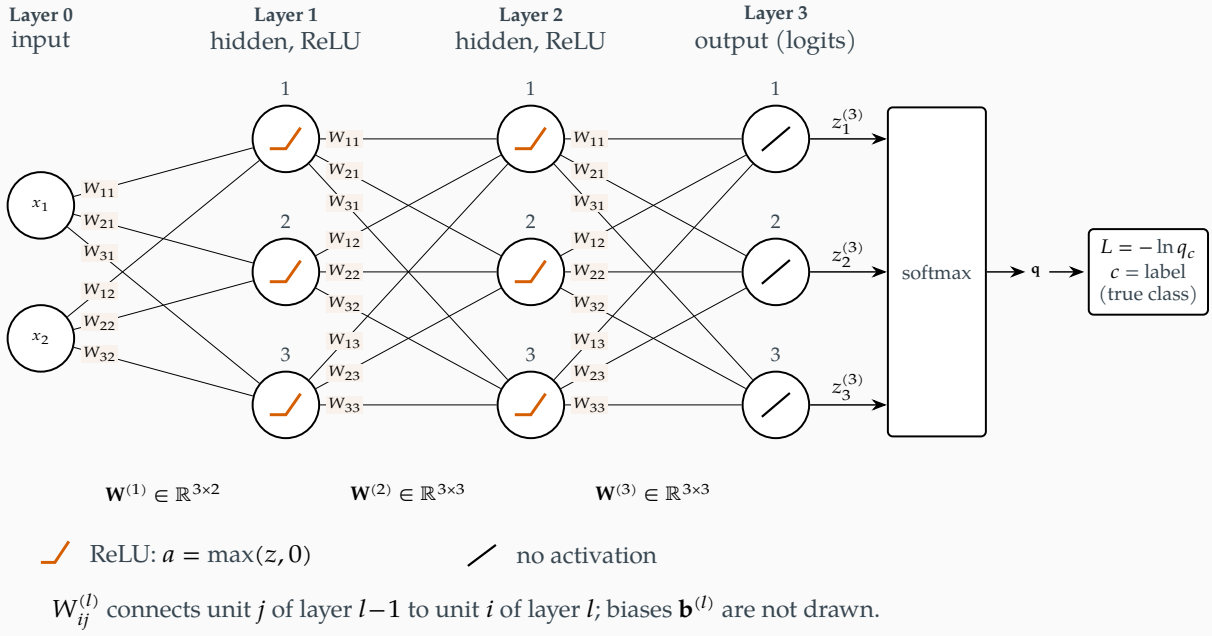


Figure 1: Example network

We assume column vectors, and define $\mathbf{a}^{(0)} = \mathbf{x}$. The last layer, Layer 3, has no activation. Its output $\mathbf{z}^{(3)}$, the *logits*, goes through the softmax function:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad l = 1, 2, 3, \tag{1.1}$$

$$\mathbf{a}^{(l)} = \mathrm{ReLU}(\mathbf{z}^{(l)}), \qquad l = 1, 2, \tag{1.2}$$

$$\mathbf{q} = \mathrm{softmax}(\mathbf{z}^{(3)}). \tag{1.3}$$

*Prepared with the assistance of Claude (Anthropic; model `claude-sonnet-5-5`), which produced the figure code, and drafts of the text in various places, and checked the numerical results with code. The author conceived the material, and reviewed and edited the text over many iterations. The author is responsible for the content.

For K classes ($K = 3$ here), the softmax is defined by

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K. \tag{1.4}$$

The weight matrix $\mathbf{W}^{(l)}$ of layer l stores the weights of each unit as a *row*: row i holds the weights into unit i of layer l, one for each unit of layer $l-1$ (see Figure 1). Thus $z_i^{(l)}$ in Eq. (1.1) is the product of row i of $\mathbf{W}^{(l)}$ with $\mathbf{a}^{(l-1)}$, and no transpose is needed in the forward pass.

For the network of Figure 1, the three weight matrices are

$$\mathbf{W}^{(1)} = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix} \in \mathbb{R}^{3\times 2},$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \end{bmatrix} \in \mathbb{R}^{3\times 3}, \qquad \mathbf{W}^{(3)} = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \end{bmatrix} \in \mathbb{R}^{3\times 3}, \tag{1.5}$$

where, as in the figure, the layer index is left off the entries: W_{ij} in $\mathbf{W}^{(l)}$ stands for $W_{ij}^{(l)}$. Row i lists the weights into unit i of layer l, and column j the weights leaving unit j of layer $l-1$. For example, row 1 of $\mathbf{W}^{(2)}$ holds W_{11}, W_{12}, W_{13}, the weights from units 1, 2 and 3 of Layer 1 into unit 1 of Layer 2. Each layer also has a bias vector $\mathbf{b}^{(l)} \in \mathbb{R}^3$, one entry per unit.

Each example has a *label* c, the index of its true class, and a *target* $\mathbf{y}$, the one-hot vector with $y_c = 1$ and all other entries 0. The loss is $L = -\ln q_c$.

There are 24 weights ($6 + 9 + 9$) and 9 biases ($3 + 3 + 3$), so 33 parameters in all, collected in θ.

Training adjusts θ by gradient descent with learning rate η, $\theta \leftarrow \theta - \eta \nabla_\theta L$. *Backpropagation* is the method for computing all 33 partial derivatives at once, in two passes: a forward pass that stores intermediate values, followed by a backward sweep that reuses them.

In these notes, Part I derives the method in general, and Part II demonstrates an example.

2 From likelihood to the cross-entropy loss

Training chooses the parameters θ, and the likelihood conveys what the best choice is. We identify four steps that lead this idea to the loss used in practice. Each step addresses one practical problem.

Step 1: Make the observed data most probable. Given a set of observed data points $\mathcal{D} = \{d_1, d_2, \dots, d_n\}$, assumed to be independent and identically distributed (i.i.d.), the **likelihood** $\mathcal{L}(\theta; \mathcal{D})$ is the probability of observing $\mathcal{D}$ under the model with parameters θ. It is regarded as a function of θ with the data held fixed.

By independence it factorises into a product:

$$\mathcal{L}(\theta; \mathcal{D}) = P(\mathcal{D} \mid \theta) = \prod_{i=1}^{n} P(d_i \mid \theta). \tag{2.1}$$

Training searches for the θ that makes $\mathcal{L}$ as large as possible, improving θ step by step. (For a neural network this finds a good local maximum rather than a guaranteed global one)

Step 2: Take the logarithm. Maximising Eq. (2.1) directly runs into two practical problems. First, every factor is at most 1, so the product shrinks geometrically with n. For instance, for 1000 examples each given probability 0.1 the computed product is exactly 0. Second, differentiating a product of n probabilities with the product rule gives a sum of n terms, each a product of $n-1$ probabilities and one derivative:

$$\begin{aligned}\frac{d}{dx}(f_1(x)f_2(x)\cdots f_n(x)) = {} & f_1'(x)f_2(x)\cdots f_n(x) + f_1(x)f_2'(x)\cdots f_n(x) \\ & + \cdots + f_1(x)f_2(x)\cdots f_n'(x).\end{aligned}$$

The natural logarithm removes both problems. It is strictly increasing, so it does not move the maximiser, and because $\ln(ab) = \ln a + \ln b$ it turns the product into a sum:

$$\ell(\theta; \mathcal{D}) = \ln \mathcal{L}(\theta; \mathcal{D}) = \sum_{i=1}^{n} \ln P(d_i \mid \theta). \tag{2.2}$$

In the example above, $\ell = (1000 * \ln 0.1) \approx -2302.6$, an ordinary number, and the derivative of ℓ is a sum of n simple terms.

Step 3: Negate, so that we minimise. The optimisers in deep learning libraries are written to *minimise*, by gradient descent. Maximising ℓ is the same as minimising $-\ell$, the *negative log-likelihood*:

$$-\ell(\theta; \mathcal{D}) = -\sum_{i=1}^{n} \ln P(d_i \mid \theta). \tag{2.3}$$

Step 4: Apply it to classification with K classes. In classification each data point is a labelled example $d_i = (\mathbf{x}_i, c_i)$, with label $c_i \in \{1, \ldots, K\}$. The model predicts the label from the input. The model says nothing about how the inputs arise in the first place. Therefore, we consider the inputs as given, use $P(c_i \mid \mathbf{x}_i, \theta)$ in place of $P(d_i \mid \theta)$, and Steps 1–3 carry over unchanged.

Let $\mathbf{z}_i$ be the logits for $\mathbf{x}_i$, $\mathbf{q}_i = \text{softmax}(\mathbf{z}_i)$ the prediction (Eq. (1.4)) and $\mathbf{y}_i$ the one-hot target of the label c_i, with entries q_{ik} and y_{ik}. The probability of the observed label is entry c_i of $\mathbf{q}_i$, written q_{c_i}, and the target lets us write it as a product over all K classes:

$$P(c_i \mid \mathbf{x}_i, \theta) = q_{c_i} = \prod_{k=1}^{K} q_{ik}^{y_{ik}}. \tag{2.4}$$

Every factor with $y_{ik} = 0$ equals $q_{ik}^0 = 1$, which leaves only the factor for the true class.

A short digression. Stating facts in simple language. For the i-th example, the network turns the input $\mathbf{x}_i$ into logits $\mathbf{z}_i$. The softmax (Eq. (1.4)) turns these logits into the predicted probabilities $\mathbf{q}_i$. The target $\mathbf{y}_i$ is the one-hot vector for the label c_i. We write q_{ik} and y_{ik} for the k-th entries of $\mathbf{q}_i$ and $\mathbf{y}_i$.

"For example i, the input goes in and the network produces scores, the logits. The softmax turns the scores into probabilities. The target marks the correct class with a 1. q_ik and y_ik are entry k of each."

Back to classification with K classes. Taking the logarithm and summing over the examples, as in Eq. (2.2),

$$\ell(\theta; \mathcal{D}) = \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \ln q_{ik} = \sum_{i=1}^{n} \ln q_{c_i}. \tag{2.5}$$

The inner sum is minus the cross-entropy $H(\mathbf{y}_i, \mathbf{q}_i) = -\sum_k y_{ik} \ln q_{ik}$ (Table 2). Negating and averaging over the n examples gives the mean loss minimised in training:

$$\bar{L}(\theta) = -\frac{1}{n} \ell(\theta; \mathcal{D}) = \frac{1}{n} \sum_{i=1}^{n} H(\mathbf{y}_i, \mathbf{q}_i) = \frac{1}{n} \sum_{i=1}^{n} (-\ln q_{c_i}). \tag{2.6}$$

Minimising $\bar{L}$ therefore maximises the likelihood. For a single example the loss is $L = -\ln q_c$, the per-example negative log-likelihood of the observed label, which is the starting point of Section 3.

The binary case is the logistic loss. With $K = 2$ the two logits carry one redundant degree of freedom: adding the same constant to both leaves $\mathbf{q}$ unchanged, so only their difference matters. With $z = z_1 - z_2$,

$$q_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-z}} = \sigma(z), \qquad q_2 = 1 - \sigma(z).$$

For example i the target satisfies $y_{i2} = 1 - y_{i1}$, so Eq. (2.6) becomes

$$\bar{L} = \frac{1}{n} \sum_{i=1}^{n} \Big[-y_{i1} \ln \sigma(z_i) - (1 - y_{i1}) \ln(1 - \sigma(z_i)) \Big], \tag{2.7}$$

the *logistic loss* of binary logistic regression, in which $y_{i1} \in \{0, 1\}$ plays the role of the binary label. Logistic loss is cross-entropy with two classes, and softmax with cross-entropy is its extension to $K \geq 3$.

Deep networks use the same loss. Nothing above depends on how the logits are computed. In our network they are $\mathbf{z}^{(3)}$, the output of the last linear layer (Eq. (1.1)); the ReLU hidden layers only change how $\mathbf{z}^{(3)}$ depends on $\mathbf{x}$. The probability model, the likelihood and the loss keep the same form.

The chain in one line.

$$\underbrace{\max_{\theta} \mathcal{L}}_{\text{likelihood}} \iff \underbrace{\max_{\theta} \ell}_{\text{a stable sum}} \iff \underbrace{\min_{\theta} (-\ell)}_{\text{for gradient descent}} \iff \underbrace{\min_{\theta} \bar{L}}_{\text{mean cross-entropy}}, \tag{2.8}$$

where the last step holds because $-\ell = n\bar{L}$ and n is a positive constant.

Note. This development follows S. Raschka's FAQ entry on negative log-likelihood and logistic loss,[1] extended here from the binary sigmoid case to K classes with softmax; the wording is our own.

3 The loss and its gradient at the logits

For one example with label c, the loss is the per-example negative log-likelihood $L = -\ln q_c$ of Section 2 (Eq. (2.6)). This section first looks at its value for an untrained network and then derives its gradient with respect to the logits, which starts the backward pass.

The loss at initialisation. A network with small random weights outputs a nearly uniform $\mathbf{q}$, so its initial loss is close to $\ln K$, where K is the number of classes.

Derivation of the initial loss $\ln K$*:*

1. **Near-equal logits:** The logits are $\mathbf{z}^{(3)} = \mathbf{W}^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)}$ (Eq. (1.1) with $l = 3$), where $\mathbf{a}^{(2)}$ holds the activations of Layer 2. With small random $\mathbf{W}^{(3)}$, $\mathbf{b}^{(3)} = \mathbf{0}$ and activations of moderate size, every logit is close to zero. Writing z_k for $z_k^{(3)}$,

   $$z_k \approx 0 \quad \forall k \in \{1, \dots, K\}.$$

   In fact, it suffices that $z_1 \approx \cdots \approx z_K$, since softmax is invariant to adding a constant to every logit.
2. **Uniform probabilities:** If all logits equal a common value a, the softmax gives

   $$q_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} = \frac{e^{a}}{Ke^{a}} = \frac{1}{K} \quad \forall k,$$

   i.e. the uniform distribution over the K classes.
3. **Loss:** Substituting $q_c = 1/K$ into the per-example loss for the true class c gives

   $$L = -\ln q_c = -\ln\left(\frac{1}{K}\right) = \ln K.$$

Sanity check. For $K = 10$ the initial loss should be about $\ln 10 \approx 2.303$. A much larger value usually indicates a bug or an overly large weight initialization.

When the loss is calculated with the natural logarithm, *nat* is the conventional unit. We may say, for instance, "the loss is 2.303 nats".

[1] https://sebastianraschka.com/faq/docs/negative-log-likelihood-logistic-loss.html

The loss in terms of the logits. Write $\mathbf{z}$ for $\mathbf{z}^{(3)}$. By the definition of the softmax, Eq. (1.4), $q_c = e^{z_c} / \sum_j e^{z_j}$, so

$$\begin{aligned} L &= -\ln q_c && (3.1)\\ &= -\ln \frac{e^{z_c}}{\sum_j e^{z_j}} \\ &= -\left(\ln e^{z_c} - \ln \sum_j e^{z_j} \right) \\ &= -z_c + \ln \sum_j e^{z_j}\,, && (3.2) \end{aligned}$$

where j ranges over the K classes (the $K = 3$ output units), and c is the fixed label.

The last line, Eq. (3.2), is how libraries compute L (a log-sum-exp), because it avoids taking the logarithm of a tiny probability. Note that $L \geq 0$, because $\sum_j e^{z_j} \geq e^{z_c}$.

Differentiating term by term. Let k range over the K classes. We differentiate the two terms of Eq. (3.2) with respect to the logit z_k separately.

The first term, $-z_c$. It depends on z_k only when $k = c$, so its derivative is -1 if $k = c$ and 0 otherwise. We write this as $-[k{=}c]$, where $[k{=}c]$ is 1 if $k = c$ and 0 otherwise. This is exactly the one-hot target, so the derivative of the first term is $-y_k$.

The second term, $\ln \sum_j e^{z_j}$. The chain rule is needed here. Let

$$u = \sum_j e^{z_j} = e^{z_1} + e^{z_2} + \cdots + e^{z_K}. \tag{3.3}$$

Applying the chain rule, we get

$$\begin{aligned} \frac{\partial}{\partial z_k} \left(\ln \sum_j e^{z_j} \right) &= \frac{\partial}{\partial z_k} (\ln u) \\ &= \frac{\partial \ln u}{\partial u} \cdot \frac{\partial u}{\partial z_k} \\ &= \frac{1}{u} \cdot \frac{\partial u}{\partial z_k} \\ &= \frac{1}{\sum_j e^{z_j}} \cdot \frac{\partial}{\partial z_k} \left(\sum_j e^{z_j} \right). && (3.4) \end{aligned}$$

It remains to differentiate every term of the sum with respect to z_k:

$$\frac{\partial}{\partial z_k} \left(\sum_j e^{z_j} \right) = \frac{\partial}{\partial z_k} \left(e^{z_1} + e^{z_2} + \cdots + e^{z_K} \right). \tag{3.5}$$

Note that

- for any term with $j \neq k$ (such as e^{z_1}, e^{z_2}, etc.), z_j is treated as a constant, and the derivative of a constant is 0;
- for the term with $j = k$, the derivative of e^{z_k} with respect to z_k is simply e^{z_k}.

Therefore, out of the entire sum, only one term survives:

$$\frac{\partial}{\partial z_k}\left(\sum_j e^{z_j}\right) = 0 + 0 + \cdots + e^{z_k} + \cdots + 0 = e^{z_k}. \tag{3.6}$$

Substituting this into Eq. (3.4),

$$\frac{\partial}{\partial z_k}\left(\ln \sum_j e^{z_j}\right) = \left(\frac{1}{\sum_j e^{z_j}}\right) \cdot e^{z_k} = \frac{e^{z_k}}{\sum_j e^{z_j}} = q_k, \tag{3.7}$$

the k-th entry of the softmax, Eq. (1.4).

The gradient at the logits. Adding the derivatives of the two terms,

$$\frac{\partial L}{\partial z_k} = -[k{=}c] + \frac{e^{z_k}}{\sum_j e^{z_j}} = -y_k + q_k = q_k - y_k\,. \tag{3.8}$$

That is, the k-th entry of the gradient is the k-th entry of the predicted distribution $\mathbf{q}$ minus the k-th entry of the target $\mathbf{y}$. In vector form,

$$\delta^{(3)} := \frac{\partial L}{\partial \mathbf{z}^{(3)}} = \mathbf{q} - \mathbf{y}\,. \tag{3.9}$$

Remarks. Two remarks on the gradient $\delta^{(3)} = \mathbf{q} - \mathbf{y}$ at the logits:

- The entries sum to zero, since $\sum_k q_k = \sum_k y_k = 1$. This reflects the fact that softmax is unchanged when the same constant is added to every logit, so moving all logits together cannot change the loss.
- Each entry lies in $[-1, 1]$: the true class has $\delta_c = q_c - 1 \in [-1, 0]$, every other class has $\delta_k = q_k \in [0, 1]$. A gradient-descent step on the logits would raise the true logit in proportion to the probability still missing, $1 - q_c$, and lower each wrong logit in proportion to the probability it holds.

4 Backpropagation through a layer

Consider layer l with input $\mathbf{a}^{(l-1)}$, and suppose $\delta^{(l)} = \partial L / \partial \mathbf{z}^{(l)}$ is known. Three results follow from the chain rule.

Gradient of the weights and biases. The pre-activation $z_i^{(l)} = \sum_j W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$ depends on $W_{ij}^{(l)}$ only through the single term $W_{ij}^{(l)} a_j^{(l-1)}$, and no other pre-activation $z_{i'}^{(l)}$ $(i' \neq i)$ involves $W_{ij}^{(l)}$, so by the chain rule only one term survives:

$$\frac{\partial L}{\partial W_{ij}^{(l)}} = \frac{\partial L}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l)}\, a_j^{(l-1)}. \tag{4.1}$$

The bias $b_i^{(l)}$ likewise enters only $z_i^{(l)}$, as a plain additive term, so $\partial z_i^{(l)}/\partial b_i^{(l)} = 1$ and

$$\frac{\partial L}{\partial b_i^{(l)}} = \frac{\partial L}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = \delta_i^{(l)} \cdot 1 = \delta_i^{(l)}. \tag{4.2}$$

A bias behaves like a weight on an input that is always 1. In matrix form, $\nabla_{\mathbf{W}^{(l)}} L = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top$, an outer product, and $\nabla_{\mathbf{b}^{(l)}} L = \boldsymbol{\delta}^{(l)}$.

Gradient passed to the layer below. The entry $a_j^{(l-1)}$ influences the loss through every $z_i^{(l)}$, so the contributions add:

$$\frac{\partial L}{\partial a_j^{(l-1)}} = \sum_i \frac{\partial L}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial a_j^{(l-1)}} = \sum_i \delta_i^{(l)} W_{ij}^{(l)} = \left((\mathbf{W}^{(l)})^\top \boldsymbol{\delta}^{(l)} \right)_j. \tag{4.3}$$

Through the activation. Since $a_j^{(l-1)} = f(z_j^{(l-1)})$, where f is the activation function (ReLU in layers 1 and 2) and acts elementwise, $\partial L/\partial z_j^{(l-1)} = (\partial L/\partial a_j^{(l-1)}) f'(z_j^{(l-1)})$, and in vector form

$$\boldsymbol{\delta}^{(l-1)} = \left((\mathbf{W}^{(l)})^\top \boldsymbol{\delta}^{(l)} \right) \odot f'(\mathbf{z}^{(l-1)}). \tag{4.4}$$

The backward pass is therefore: start from $\boldsymbol{\delta}^{(3)} = \mathbf{q} - \mathbf{y}$ (Eq. (3.9)); at each layer form the weight gradient as an outer product (Eq. (4.1)), multiply by $(\mathbf{W}^{(l)})^\top$ (Eq. (4.3)), and multiply elementwise by f' of the stored pre-activation (Eq. (4.4)). Shapes provide a check: with n_l the number of units in layer l, $\boldsymbol{\delta}^{(l)} \in \mathbb{R}^{n_l}$ and $\mathbf{a}^{(l-1)} \in \mathbb{R}^{n_{l-1}}$ give $\nabla_{\mathbf{W}^{(l)}} L \in \mathbb{R}^{n_l \times n_{l-1}}$, the shape of $\mathbf{W}^{(l)}$.

What the forward pass must store. For every layer, the input $\mathbf{a}^{(l-1)}$ (for the weight gradient) and $\mathbf{z}^{(l)}$ (for f'). The memory needed grows with depth and width, which is one reason why training a large network needs much more memory than running it.

Cost. Each layer costs about one matrix–vector product in the forward pass, and about two operations of the same size in the backward pass (the outer product for the weight gradient and the product with $\mathbf{W}^\top$). A backward sweep therefore costs roughly twice a forward pass, whatever the number of parameters, whereas finite differences need two forward passes per parameter. That is what makes backpropagation practical.

5 The ReLU

$\mathrm{ReLU}(z) = \max(z, 0)$ has $\mathrm{ReLU}'(z) = 1$ for $z > 0$ and 0 for $z < 0$. At $z = 0$ it has a kink; the convention here is $\mathrm{ReLU}'(0) = 0$. A hidden unit with $z \le 0$ is *inactive*, and:

- its δ is zero, so the weights feeding it (a row of $\nabla_{\mathbf{W}^{(l)}} L$) and its bias receive no gradient;
- its output is $a = 0$, so the weights leaving it (a column of the next layer's weight gradient, $\boldsymbol{\delta}\, \mathbf{a}^\top$) receive none either;

- a unit that is inactive for *every* training example is never updated and cannot recover; it is called *dead*.

6 Checking a gradient numerically

A derived gradient can be tested against the definition of a derivative. The *central difference*

$$\frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon} = \frac{\partial L}{\partial \theta_i} + \frac{\varepsilon^2}{6}\frac{\partial^3 L}{\partial \theta_i^3} + \cdots \tag{6.1}$$

has an error of order ε^2 (the one-sided difference has order ε), which is why it is preferred. The step ε is a compromise: too large and the truncation error dominates; too small and the subtraction of two nearly equal losses loses digits to rounding. In double precision a value around 10^{-5} or 10^{-6} is typical. In a hand calculation with $\varepsilon = 0.01$, the losses must be carried to six decimals.

Each parameter needs two forward passes, so checking all N parameters costs $2N$ passes against a single backward pass for backpropagation. This is why the check is a test of the code or the algebra, never the way a network is trained.

7 The update, epochs and batches

For a gradient $\mathbf{g} = \nabla_\theta L$, the first-order expansion gives

$$L(\theta - \eta\mathbf{g}) = L(\theta) - \eta\,\|\mathbf{g}\|^2 + O(\eta^2), \tag{7.1}$$

so for small enough η the loss decreases unless $\mathbf{g} = \mathbf{0}$. The $O(\eta^2)$ term involves curvature and explains why a large step can overshoot.

Epochs and batches. The training data consist of n labelled examples, with per-example losses $L_1, \ldots, L_n$. One pass over all n examples is an *epoch*; training usually runs for many epochs, often reshuffling the examples before each one. How many updates an epoch contains depends on how many examples are used for each update:

- *Per-example* (stochastic) gradient descent updates θ after every single example, so an epoch has n updates.
- With *minibatches* of size B, each update uses the gradient of the *mean* loss over the batch, so an epoch has n/B updates (rounded up if B does not divide n).
- With $B = n$ (*full-batch* gradient descent), each update uses the mean loss over the whole data set, so an epoch has a single update.

By linearity of differentiation, the gradient of the mean loss over a batch is the mean of the per-example gradients. Small batches give noisier but more frequent updates; large batches give smoother gradient estimates and make efficient use of matrix hardware.

A batch as a matrix. With $B = 5$ examples, the inputs are the columns of a matrix $\mathbf{X} \in \mathbb{R}^{2\times 5}$ and the targets are the columns of a matrix $\mathbf{Y} \in \mathbb{R}^{3\times 5}$; column b of $\mathbf{Y}$ is the one-hot target of column b of $\mathbf{X}$. With labels $c = (2, 3, 1, 2, 3)$,

$$\mathbf{X} = \begin{pmatrix} 1 & 2 & 0 & -1 & 3 \\ 2 & 1 & 1 & 2 & -1 \end{pmatrix}, \qquad \mathbf{Y} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

Column b of $\mathbf{X}$ is $\mathbf{x}_b$ and column b of $\mathbf{Y}$ is $\mathbf{y}_b$, so the first column of each is the pair $\mathbf{x} = (1, 2)^\top$, $\mathbf{y} = (0, 1, 0)^\top$ of the exercise. The forward pass keeps the same shapes with one column per example: $\mathbf{A}^{(0)} = \mathbf{X}$, and $\mathbf{Z}^{(l)}$, $\mathbf{A}^{(l)}$, the output $\mathbf{Q}$ and $\mathbf{\Delta}^{(3)} = \mathbf{Q} - \mathbf{Y}$ each have 5 columns. In general, if the examples of a batch are the columns of $\mathbf{A}^{(l-1)}$ and the matching δ's the columns of $\mathbf{\Delta}^{(l)}$, the weight gradient is $\mathbf{\Delta}^{(l)}(\mathbf{A}^{(l-1)})^\top$ (divided by B for the mean loss): a batch is more columns, and the same products. None of this depends on the architecture. It holds for any feed-forward network trained by backpropagation; only the layer sizes and the activation functions differ.

One update per batch. The columns of $\mathbf{\Delta}^{(l)}$ are error signals, one per example; they are not instructions to update the weights one at a time. For the batch of five examples above, backpropagation computes $\mathbf{\Delta}^{(3)}$, $\mathbf{\Delta}^{(2)}$ and $\mathbf{\Delta}^{(1)}$ in that order, from the output layer towards the input, using the same weights as the forward pass. At each layer it combines the five columns into one gradient for the mean loss:

$$\nabla_{\mathbf{W}^{(l)}} L = \frac{1}{B}\mathbf{\Delta}^{(l)}(\mathbf{A}^{(l-1)})^\top = \frac{1}{B}\sum_{b=1}^{B} \delta_b^{(l)}(\mathbf{a}_b^{(l-1)})^\top, \qquad \nabla_{\mathbf{b}^{(l)}} L = \frac{1}{B}\sum_{b=1}^{B} \delta_b^{(l)}, \tag{7.2}$$

with $B = 5$. The weight gradient is the average of the per-example gradients of Eq. (4.1), and the bias gradient is the average of the deltas. Only after all the gradients have been computed does the optimiser update each layer, $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \nabla_{\mathbf{W}^{(l)}} L$ and $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \nabla_{\mathbf{b}^{(l)}} L$. So there is one update per minibatch, not one per example or per entry of $\mathbf{\Delta}^{(l)}$. The backward calculation has a layer order; the update does not follow the order of the entries in $\mathbf{\Delta}^{(l)}$.

Since the per-example outer products are summed, reordering the examples consistently in both $\mathbf{\Delta}^{(l)}$ and $\mathbf{A}^{(l-1)}$ does not change the gradient. Reordering the columns means multiplying both matrices on the right by the same permutation matrix $\mathbf{P}$, and $\mathbf{P}\mathbf{P}^\top = \mathbf{I}$, so

$$(\mathbf{\Delta}^{(l)}\mathbf{P})(\mathbf{A}^{(l-1)}\mathbf{P})^\top = \mathbf{\Delta}^{(l)}\mathbf{P}\mathbf{P}^\top(\mathbf{A}^{(l-1)})^\top = \mathbf{\Delta}^{(l)}(\mathbf{A}^{(l-1)})^\top.$$

Reordering only one of the two matrices would pair each delta with the wrong example, and the gradient would change.

What practice adds. Large-scale training typically uses large minibatches, an adaptive optimizer such as Adam in place of the plain step, a learning-rate schedule and gradient clipping. The gradient formulas above are unchanged; only what is done with the gradient differs.

8 Summary of formulas

- **Forward:** $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{a}^{(l)} = \mathrm{ReLU}(\mathbf{z}^{(l)})$, $\mathbf{q} = \mathrm{softmax}(\mathbf{z}^{(3)})$, $L = -\ln q_c$ (Eqs. (1.1)–(1.3), (3.1)).
- **Start of the backward pass:** $\delta^{(3)} = \mathbf{q} - \mathbf{y}$ (Eq. (3.9)).
- **Each layer:**

$$\nabla_{\mathbf{W}^{(l)}} L = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top,$$
$$\nabla_{\mathbf{b}^{(l)}} L = \delta^{(l)},$$
$$\delta^{(l-1)} = ((\mathbf{W}^{(l)})^\top \delta^{(l)}) \odot \mathrm{ReLU}'(\mathbf{z}^{(l-1)}).$$

  See Eqs. (4.1), (4.2), and (4.4).
- **Check:** Central difference with step ε (Eq. (6.1)).
- **Update:** $\theta \leftarrow \theta - \eta \nabla_\theta L$.

9 Differences from PyTorch

Code written with PyTorch computes the same numbers as our notes here, but it does not look exactly like the formulas above. The weights of a layer are laid out as in the notes, whereas a batch is laid out the other way round, and the transposes in the code come from that.

Table 1 lists the differences.

	Our notes	**PyTorch**
Weights of a layer	$\mathbf{W}^{(l)}$ of shape $n_l \times n_{l-1}$; row i holds the weights into unit i	`weight` of `nn.Linear`, shape `(out_features, in_features)`: the same layout
Bias	$\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$	`bias`, shape `(out_features,)`
A batch	examples are the *columns* of $\mathbf{A}^{(l-1)}$, shape $n_{l-1} \times B$	examples are the *rows* of `X`, shape `(B, in_features)`
A layer	$\mathbf{Z}^{(l)} = \mathbf{W}^{(l)}\mathbf{A}^{(l-1)} + \mathbf{b}^{(l)}$ (Eq. (1.1))	`Z = X @ weight.T + bias`
Weight gradient	$\mathbf{\Delta}^{(l)}(\mathbf{A}^{(l-1)})^\top$, divided by B for the mean loss	`weight.grad`, same shape as `weight`; includes the $1/B$ of the default mean loss
Loss	$L = -\ln q_c$, computed as $-z_c + \ln \sum_j e^{z_j}$ (Eq. (3.2))	`nn.CrossEntropyLoss` takes the logits $\mathbf{z}^{(3)}$, not $\mathbf{q}$, so the network does not end in a softmax layer
Label	$c \in \{1, 2, 3\}$; target $\mathbf{y}$ one-hot	class index in $\{0, 1, 2\}$, no one-hot vector needed

Table 1: Notes versus PyTorch conventions

The transpose in `X @ weight.T` appears because examples are stored as rows; the weight matrix itself is the same $\mathbf{W}^{(l)}$. When comparing code with these notes, read `weight` as $\mathbf{W}^{(l)}$ and `X` as $(\mathbf{A}^{(l-1)})^\top$. Remember too that classes are numbered from 0 in code, so the label $c = 2$ of the worked example is the index `1`.

Part II: Worked computation

10 Data

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0.5 & 0.25 \\ 0.4 & -0.1 \\ -0.3 & -0.1 \end{pmatrix}, \qquad \mathbf{W}^{(2)} = \begin{pmatrix} 0.5 & -0.5 & 0.3 \\ 0.2 & 0.4 & -0.1 \\ 0.1 & 0.6 & 0.5 \end{pmatrix},$$

$$\mathbf{W}^{(3)} = \begin{pmatrix} 0.3 & -0.2 & 0.5 \\ -0.4 & 0.6 & 0.1 \\ 0.2 & 0.1 & -0.3 \end{pmatrix}, \qquad \mathbf{b}^{(1)} = \mathbf{b}^{(2)} = \mathbf{b}^{(3)} = \mathbf{0}.$$

Input $\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, label $c = 2$ (so the target is $\mathbf{y} = (0, 1, 0)^\top$), learning rate $\eta = 0.1$.

11 Forward pass and loss (Tasks 1 and 2)

Layer 1, using Eq. (1.1) (the biases are zero):

$$\begin{aligned} z_1^{(1)} &= 0.5 \cdot 1 + 0.25 \cdot 2 = 1.0000 \\ z_2^{(1)} &= 0.4 \cdot 1 + (-0.1) \cdot 2 = 0.2000 \\ z_3^{(1)} &= (-0.3) \cdot 1 + (-0.1) \cdot 2 = -0.5000 \end{aligned} \tag{11.1}$$

Applying the ReLU gives $\mathbf{a}^{(1)} = \begin{pmatrix} 1.0000 \\ 0.2000 \\ 0 \end{pmatrix}$: unit 3 is inactive because $z_3^{(1)} = -0.5 < 0$.

Layer 2:

$$\begin{aligned} z_1^{(2)} &= 0.5 \cdot 1 + (-0.5) \cdot 0.2 + 0.3 \cdot 0 = 0.4000 \\ z_2^{(2)} &= 0.2 \cdot 1 + 0.4 \cdot 0.2 + (-0.1) \cdot 0 = 0.2800 \\ z_3^{(2)} &= 0.1 \cdot 1 + 0.6 \cdot 0.2 + 0.5 \cdot 0 = 0.2200 \end{aligned} \tag{11.2}$$

Because $a_3^{(1)} = 0$, the third column of $\mathbf{W}^{(2)}$ had no effect here. All three values are positive, so $\mathbf{a}^{(2)} = \mathbf{z}^{(2)} = \begin{pmatrix} 0.4000 \\ 0.2800 \\ 0.2200 \end{pmatrix}$.

Layer 3 (the logits):

$$\begin{aligned} z_1^{(3)} &= 0.3 \cdot 0.4 + (-0.2) \cdot 0.28 + 0.5 \cdot 0.22 = 0.1740 \\ z_2^{(3)} &= (-0.4) \cdot 0.4 + 0.6 \cdot 0.28 + 0.1 \cdot 0.22 = 0.0300 \\ z_3^{(3)} &= 0.2 \cdot 0.4 + 0.1 \cdot 0.28 + (-0.3) \cdot 0.22 = 0.0420 \end{aligned} \tag{11.3}$$

Softmax, Eq. (1.4):

$$\begin{aligned} e^{z_1^{(3)}} &= e^{0.1740} = 1.1901 \\ e^{z_2^{(3)}} &= e^{0.0300} = 1.0305 \\ e^{z_3^{(3)}} &= e^{0.0420} = 1.0429 \\ \textstyle\sum_j e^{z_j^{(3)}} &= 3.2634 \\ q_1 &= 1.1901/3.2634 = 0.3647 \\ q_2 &= 1.0305/3.2634 = 0.3158 \\ q_3 &= 1.0429/3.2634 = 0.3196 \end{aligned} \tag{11.4}$$

Loss, Eq. (3.1):

$$L = -\ln q_2 = -\ln 0.3158 = 1.1528 \tag{11.5}$$

This is slightly above $\ln 3 = 1.0986$ because the true class received $q_2 = 0.3158 < 1/3$.

12 Gradient at the logits (Task 3)

By Eq. (3.8),

$$\begin{aligned} \delta_1^{(3)} &= q_1 - y_1 = 0.3647 - 0 = 0.3647 \\ \delta_2^{(3)} &= q_2 - y_2 = 0.3158 - 1 = -0.6842 \\ \delta_3^{(3)} &= q_3 - y_3 = 0.3196 - 0 = 0.3196 \end{aligned} \tag{12.1}$$

so $\boldsymbol{\delta}^{(3)} = \begin{pmatrix} 0.3647 \\ -0.6842 \\ 0.3196 \end{pmatrix}$, and its entries sum to zero.

13 Backward pass (Task 4)

Layer 3. By Eqs. (4.1) and (4.2),

$$\nabla_{\mathbf{W}^{(3)}} L = \boldsymbol{\delta}^{(3)} (\mathbf{a}^{(2)})^\top = \begin{pmatrix} 0.3647 \\ -0.6842 \\ 0.3196 \end{pmatrix} \begin{pmatrix} 0.4 & 0.28 & 0.22 \end{pmatrix} = \begin{pmatrix} 0.1459 & 0.1021 & 0.0802 \\ -0.2737 & -0.1916 & -0.1505 \\ 0.1278 & 0.0895 & 0.0703 \end{pmatrix}, \tag{13.1}$$

$$\nabla_{\mathbf{b}^{(3)}} L = \boldsymbol{\delta}^{(3)} = \begin{pmatrix} 0.3647 \\ -0.6842 \\ 0.3196 \end{pmatrix}. \tag{13.2}$$

Layer 2. First pass the gradient down through $(\mathbf{W}^{(3)})^\top$ (Eq. (4.3)):

$$\begin{aligned} \delta_1^{(2)} &= 0.3 \cdot 0.3647 + (-0.4) \cdot (-0.6842) + 0.2 \cdot 0.3196 = 0.4470 \\ \delta_2^{(2)} &= (-0.2) \cdot 0.3647 + 0.6 \cdot (-0.6842) + 0.1 \cdot 0.3196 = -0.4515 \\ \delta_3^{(2)} &= 0.5 \cdot 0.3647 + 0.1 \cdot (-0.6842) + (-0.3) \cdot 0.3196 = 0.0180 \end{aligned} \tag{13.3}$$

All second-layer units are active, $\mathrm{ReLU}'(\mathbf{z}^{(2)}) = \begin{pmatrix}1\\1\\1\end{pmatrix}$, so $\boldsymbol{\delta}^{(2)} = \begin{pmatrix}0.4470\\-0.4515\\0.0180\end{pmatrix}$. Then

$$\nabla_{\mathbf{W}^{(2)}} L = \boldsymbol{\delta}^{(2)} (\mathbf{a}^{(1)})^\top = \begin{pmatrix}0.4470\\-0.4515\\0.0180\end{pmatrix} \begin{pmatrix}1 & 0.2 & 0\end{pmatrix} = \begin{pmatrix}0.4470 & 0.0894 & 0\\-0.4515 & -0.0903 & 0\\0.0180 & 0.0036 & 0\end{pmatrix}, \tag{13.4}$$

and $\nabla_{\mathbf{b}^{(2)}} L = \boldsymbol{\delta}^{(2)}$. The third column is zero because $a_3^{(1)} = 0$.

Layer 1. The product with $(\mathbf{W}^{(2)})^\top$ (Eq. (4.3)) gives

$$\begin{aligned} r_1^{(1)} &= 0.5 \cdot 0.447 + 0.2 \cdot (-0.4515) + 0.1 \cdot 0.018 = 0.1350 \\ r_2^{(1)} &= (-0.5) \cdot 0.447 + 0.4 \cdot (-0.4515) + 0.6 \cdot 0.018 = -0.3933 \\ r_3^{(1)} &= 0.3 \cdot 0.447 + (-0.1) \cdot (-0.4515) + 0.5 \cdot 0.018 = 0.1883 \end{aligned} \tag{13.5}$$

Now apply the mask $\mathrm{ReLU}'(\mathbf{z}^{(1)}) = \begin{pmatrix}1\\1\\0\end{pmatrix}$ (Eq. (4.4)):

$$\boldsymbol{\delta}^{(1)} = \begin{pmatrix}0.1350\\-0.3933\\0.1883\end{pmatrix} \odot \begin{pmatrix}1\\1\\0\end{pmatrix} = \begin{pmatrix}0.1350\\-0.3933\\0\end{pmatrix}. \tag{13.6}$$

The value 0.1883 is discarded: unit 3 is inactive, so it cannot pass gradient on. Finally

$$\nabla_{\mathbf{W}^{(1)}} L = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top = \begin{pmatrix}0.1350\\-0.3933\\0\end{pmatrix} \begin{pmatrix}1 & 2\end{pmatrix} = \begin{pmatrix}0.1350 & 0.2700\\-0.3933 & -0.7866\\0 & 0\end{pmatrix}, \tag{13.7}$$

$$\nabla_{\mathbf{b}^{(1)}} L = \boldsymbol{\delta}^{(1)} = \begin{pmatrix}0.1350\\-0.3933\\0\end{pmatrix}. \tag{13.8}$$

The third row of $\nabla_{\mathbf{W}^{(1)}} L$ is zero, for the reason given above.

14 Numerical check (Task 5)

Take $W_{12}^{(1)} = 0.25$, the weight from x_2 into hidden unit 1, with $\varepsilon = 0.01$. This entry is a good test because it influences the loss through all later layers, so it exercises the whole backward chain. Two extra forward passes give

$$L(+\varepsilon) = 1.155473, \qquad L(-\varepsilon) = 1.150073, \tag{14.1}$$

so the central difference of Eq. (6.1) is

$$\frac{L(+\varepsilon) - L(-\varepsilon)}{2\varepsilon} = 0.270010, \tag{14.2}$$

(or 0.27000 if the printed six-decimal losses are used), against 0.270010 from backpropagation, the $(1,2)$ entry of $\nabla_{\mathbf{W}^{(1)}} L$ in Eq. (13.7). All 33 gradient entries were checked the same way in double precision, with a largest discrepancy below 10^{-9}.

15 One update (Task 6)

With $\eta = 0.1$, the first layer becomes

$$\mathbf{W}^{(1)} \leftarrow \begin{pmatrix} 0.5 & 0.25 \\ 0.4 & -0.1 \\ -0.3 & -0.1 \end{pmatrix} - 0.1 \begin{pmatrix} 0.1350 & 0.2700 \\ -0.3933 & -0.7866 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 0.4865 & 0.2230 \\ 0.4393 & -0.0213 \\ -0.3000 & -0.1000 \end{pmatrix}, \tag{15.1}$$

$$\mathbf{b}^{(1)} \leftarrow \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} - 0.1 \begin{pmatrix} 0.1350 \\ -0.3933 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.0135 \\ 0.0393 \\ 0 \end{pmatrix}. \tag{15.2}$$

The other layers are updated in the same way:

$$\mathbf{W}^{(2)} \leftarrow \begin{pmatrix} 0.4553 & -0.5089 & 0.3000 \\ 0.2452 & 0.4090 & -0.1000 \\ 0.0982 & 0.5996 & 0.5000 \end{pmatrix}, \qquad \mathbf{b}^{(2)} \leftarrow \begin{pmatrix} -0.0447 \\ 0.0452 \\ -0.0018 \end{pmatrix}, \tag{15.3}$$

$$\mathbf{W}^{(3)} \leftarrow \begin{pmatrix} 0.2854 & -0.2102 & 0.4920 \\ -0.3726 & 0.6192 & 0.1151 \\ 0.1872 & 0.0911 & -0.3070 \end{pmatrix}, \qquad \mathbf{b}^{(3)} \leftarrow \begin{pmatrix} -0.0365 \\ 0.0684 \\ -0.0320 \end{pmatrix}. \tag{15.4}$$

Re-running the forward pass with the new parameters:

$$\mathbf{z}^{(1)} = \begin{pmatrix} 0.9190 \\ 0.4360 \\ -0.5000 \end{pmatrix}, \quad \mathbf{a}^{(1)} = \begin{pmatrix} 0.9190 \\ 0.4360 \\ 0 \end{pmatrix}, \quad \mathbf{z}^{(2)} = \begin{pmatrix} 0.1518 \\ 0.4488 \\ 0.3499 \end{pmatrix}, \tag{15.5}$$

$$\mathbf{a}^{(2)} = \begin{pmatrix} 0.1518 \\ 0.4488 \\ 0.3499 \end{pmatrix}, \quad \mathbf{z}^{(3)} = \begin{pmatrix} 0.0847 \\ 0.3300 \\ -0.0701 \end{pmatrix}, \quad \mathbf{q} = \begin{pmatrix} 0.3190 \\ 0.4077 \\ 0.2733 \end{pmatrix}. \tag{15.6}$$

The new loss is $L = 0.8972$, a decrease of 0.2556. The probability of the true class rose from 0.3158 to 0.4077, while the wrong classes fell (q_1 from 0.3647 to 0.3190, q_3 from 0.3196 to 0.2733). Unit 3 of the first layer is still inactive, with $z_3^{(1)} = -0.5$ unchanged: a dead unit does not move at all.

For the first-order check, Eq. (7.1), the sum of squares of all 33 gradient entries is $\|\nabla L\|^2 = 2.7666$, so $\eta\|\nabla L\|^2 = 0.2767$, against the actual decrease of 0.2556. The difference is the second-order term, and it shrinks as $\eta \to 0$.

16 A second example (Tasks 7 and 8)

For $\mathbf{x}' = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ with label $c' = 3$, under the *original* parameters:

$$\mathbf{z}^{(1)} = \begin{pmatrix} 1.2500 \\ 0.7000 \\ -0.7000 \end{pmatrix}, \quad \mathbf{a}^{(1)} = \begin{pmatrix} 1.2500 \\ 0.7000 \\ 0 \end{pmatrix}, \quad \mathbf{z}^{(2)} = \begin{pmatrix} 0.2750 \\ 0.5300 \\ 0.5450 \end{pmatrix}, \tag{16.1}$$

$$\mathbf{a}^{(2)} = \begin{pmatrix} 0.2750 \\ 0.5300 \\ 0.5450 \end{pmatrix}, \quad \mathbf{z}^{(3)} = \begin{pmatrix} 0.2490 \\ 0.2625 \\ -0.0555 \end{pmatrix}, \quad \mathbf{q}' = \begin{pmatrix} 0.3635 \\ 0.3684 \\ 0.2681 \end{pmatrix}, \tag{16.2}$$

so $L' = -\ln q'_3 = 1.3165$, and the mean loss over the two examples is 1.2346. The gradient of the mean loss is the mean of the two per-example gradients. (A batch gradient was not computed here.) Unit 3 of layer 1 is inactive for both examples; had it been inactive for *all* training data, it would be dead. With $n = 4$ training examples, per-example updates give 4 updates per epoch and minibatches of size $B = 2$ give 2.

All parameters zero. Every $\mathbf{z}$ and $\mathbf{a}$ is then zero and $\mathbf{q}$ is uniform. The weight gradients vanish, $\nabla_{\mathbf{W}^{(3)}} L = \boldsymbol{\delta}^{(3)} (\mathbf{a}^{(2)})^\top = \mathbf{0}$ because $\mathbf{a}^{(2)} = \mathbf{0}$, and $\boldsymbol{\delta}^{(2)} = \mathbf{0}$ because $\mathbf{W}^{(3)} = \mathbf{0}$. Only the output bias has a gradient,

$$\nabla_{\mathbf{b}^{(3)}} L = \begin{pmatrix} 0.3333 \\ -0.6667 \\ 0.3333 \end{pmatrix}, \tag{16.3}$$

and over later steps the hidden layers never receive one (confirmed over 50 steps): the network can learn only the class frequencies. More generally, any initialisation that makes the units of a layer identical keeps them identical, since they receive identical gradients; this is why weights are initialised randomly.

A different label. If the label were 1 instead of 2, so that $\mathbf{y} = (1, 0, 0)^\top$, the forward quantities $\mathbf{z}^{(l)}$, $\mathbf{a}^{(l)}$, $\mathbf{q}$ of Eqs. (11.1)–(11.4) would not change, because they do not depend on the label. The label enters only through the loss, $L = -\ln q_1 = 1.0088$ (Eq. (3.1)), and through $\boldsymbol{\delta}^{(3)} = \mathbf{q} - \mathbf{y} = \begin{pmatrix} -0.6353 \\ 0.3158 \\ 0.3196 \end{pmatrix}$ (Eq. (3.9)). Everything downstream of $\boldsymbol{\delta}^{(3)}$ in the backward pass would change.

17 Key Quantities at a Glance

Quantity	Definition	Common-sense meaning
Surprisal of outcome k	$h_{\mathbf{q}}(k) = -\ln q_k$	How surprised the model $\mathbf{q}$ is when outcome k happens: 0 when $q_k = 1$, growing without limit as $q_k \to 0$. Surprisals of independent outcomes add.
Entropy	$H(\mathbf{p}) = -\sum_k p_k \ln p_k$ $= \mathbb{E}_{k\sim\mathbf{p}}[-\ln p_k]$	Average surprisal of outcomes from $\mathbf{p}$, judged by $\mathbf{p}$ itself: the uncertainty built into $\mathbf{p}$. Needs no model and no data. It is 0 for a one-hot $\mathbf{p}$ and $\ln K$ for the uniform $\mathbf{p}$.
Cross-entropy	$H(\mathbf{p}, \mathbf{q}) = -\sum_k p_k \ln q_k$ $= \mathbb{E}_{k\sim\mathbf{p}}[-\ln q_k]$	Average surprisal of the model $\mathbf{q}$ when outcomes come from $\mathbf{p}$. Never below $H(\mathbf{p})$. For a one-hot $\mathbf{p}$ on the true class c, it is $-\ln q_c$, the loss for one example.
KL divergence	$\mathrm{KL}(\mathbf{p} \,\|\, \mathbf{q}) = \sum_k p_k \ln(p_k/q_k)$ $= H(\mathbf{p}, \mathbf{q}) - H(\mathbf{p})$	Extra surprisal we pay for using $\mathbf{q}$ instead of $\mathbf{p}$. Never negative, 0 only when $\mathbf{q} = \mathbf{p}$, and not symmetric in $\mathbf{p}$ and $\mathbf{q}$. For a one-hot $\mathbf{p}$, $H(\mathbf{p}) = 0$, so the KL divergence equals the loss.
Likelihood	$\prod_i q_\theta(c_i \mid x_i)$	How probable the model finds all the observed labels together (examples assumed independent). A function of θ with the data fixed, *not* a distribution over θ. Training makes it large.
Log-likelihood and NLL	$\ell(\theta) = \sum_i \ln q_\theta(c_i \mid x_i)$ $\mathrm{NLL} = -\ell(\theta)$	The log turns the product into a sum, and $\ell(\theta) \le 0$. Its negative, the negative log-likelihood (NLL), is the *total surprisal* of the observed labels.
Average cross-entropy loss	$\bar{L}(\theta) = -\ell(\theta)/N$ $= \frac{1}{N}\sum_i (-\ln q_\theta(c_i \mid x_i))$	The NLL per example: the one-hot cross-entropy averaged over the data set. Libraries report this number. Minimising it is maximum likelihood estimation.

Table 2: Surprisal, entropy, cross-entropy, KL divergence, and likelihood, and the cross-entropy loss. Here $\mathbf{p}$ is the target distribution, $\mathbf{q}$ is the model's predicted distribution, c is the true class, and sums over i run over the N examples. All logarithms are natural, so the quantities are in nats.

18 Three Facts About Cross-Entropy

In this section, $\mathbf{p}$ is the target distribution and $\mathbf{q}$ is the model's predicted distribution, as in Table 2. A sum written as $\sum_{p_k>0}$ runs only over the classes whose target probability is positive.

Fact 1: cross-entropy = entropy + KL divergence. Take a class k with $p_k > 0$. Since $\ln(p_k/q_k) = \ln p_k - \ln q_k$, we can write

$$-\ln q_k = -\ln p_k + \ln \frac{p_k}{q_k}. \tag{18.1}$$

So the surprisal of the model splits into two parts: the surprisal that the target itself assigns to class k, and an extra part that comes from $\mathbf{q}$ being different from $\mathbf{p}$. Multiply both sides by p_k and add over the classes with $p_k > 0$:

$$\underbrace{-\sum_{p_k>0} p_k \ln q_k}_{H(\mathbf{p},\mathbf{q})} = \underbrace{-\sum_{p_k>0} p_k \ln p_k}_{H(\mathbf{p})} + \underbrace{\sum_{p_k>0} p_k \ln \frac{p_k}{q_k}}_{\mathrm{KL}(\mathbf{p}\,\|\,\mathbf{q})}. \tag{18.2}$$

Classes with $p_k = 0$ get zero weight in all three sums (using the convention $0 \ln 0 = 0$), so leaving them out changes nothing.

In simple words: the loss is the uncertainty already present in the target, plus a penalty for the model being different from the target.

Fact 2: KL divergence is never negative. We need one inequality:

$$\ln t \le t - 1 \quad \text{for } t > 0, \text{ with equality only at } t = 1. \tag{18.3}$$

The straight line $t - 1$ touches the curve $\ln t$ at $t = 1$, and since the curve bends downwards, it stays below the line everywhere else. Now put $t = q_k/p_k$ for each class with $p_k > 0$:

$$-\mathrm{KL}(\mathbf{p}\,\|\,\mathbf{q}) = \sum_{p_k>0} p_k \ln \frac{q_k}{p_k} \le \sum_{p_k>0} p_k\Big(\frac{q_k}{p_k} - 1\Big) = \sum_{p_k>0} q_k - 1 \le 0. \tag{18.4}$$

The last step holds because the q_k over only *some* of the classes add up to at most 1. Hence $\mathrm{KL}(\mathbf{p}\,\|\,\mathbf{q}) \ge 0$. Both inequalities in Eq. (18.4) become equalities only when $q_k = p_k$ for every class k. Putting this into Eq. (18.2) gives

$$H(\mathbf{p},\mathbf{q}) \ge H(\mathbf{p}). \tag{18.5}$$

In simple words: the loss can never go below the entropy of the target, and it reaches that value exactly when the model matches the target. For a one-hot target, $H(\mathbf{p}) = 0$, so the smallest possible loss is 0.

Fact 3: entropy is at most $\ln K$, and a uniform model has loss $\ln K$. Let $\mathbf{u}$ be the uniform distribution over the K classes, $u_k = 1/K$. Then

$$\mathrm{KL}(\mathbf{p} \,\|\, \mathbf{u}) = \sum_{p_k>0} p_k \ln \frac{p_k}{1/K} = \sum_{p_k>0} p_k \ln p_k + \ln K \sum_{p_k>0} p_k = -H(\mathbf{p}) + \ln K. \tag{18.6}$$

Since KL is never negative (Fact 2), $H(\mathbf{p}) \le \ln K$, with equality only when $\mathbf{p}$ is itself uniform. This is the largest entropy any distribution over K classes can have.

Now let the *model* be uniform. For any target $\mathbf{p}$,

$$H(\mathbf{p}, \mathbf{u}) = -\sum_k p_k \ln \frac{1}{K} = \ln K \sum_k p_k = \ln K. \tag{18.7}$$

So a model that predicts $1/K$ for every class has loss exactly $\ln K$, whatever the target is.

Why this matters at the start of training. If a network starts with its logits all close to 0 (for example, when the last layer has small weights), the softmax gives $q_k \approx e^0/(Ke^0) = 1/K$ for every class. So $\mathbf{q}$ is nearly uniform, and by Eq. (18.7) the first loss is close to $\ln K$. In our worked example, $K = 3$, and the loss in Eq. (11.5) is $-\ln q_2 = -\ln 0.3158$, just above $\ln 3 = 1.0986$. In practice, a starting loss much larger than $\ln K$ is a useful warning that the initialisation needs a look.